

Calificación 6,25 de 10,00 (63%)

Pregunta 1

Incorrecta

Puntúa 0,00 sobre 1,00

🚩 Marcar pregunta

En P.O.O. ocurre una Delegación cuando un objeto pasa una relación de una clase a un método.

Seleccione una:

- ☒ Verdadero ❌
- ☐ Falso

La respuesta correcta es 'Falso'

Pregunta 2

Correcta

Puntúa 1,00 sobre 1,00

🚩 Marcar pregunta

Las interfaces pueden extenderse

Seleccione una:

- ☒ Verdadero ✔️
- ☐ Falso

Correcto, las interfaces pueden ser creadas a partir de la especialización de otra interfaz.

La respuesta correcta es 'Verdadero'

Pregunta 3

Incorrecta

Puntúa 0,00 sobre 1,00

🚩 Marcar pregunta

Las interfaces no tienen ningún comportamiento.

Seleccione una:

- ☒ Verdadero ❌
- ☐ Falso

La respuesta correcta es 'Falso'

Pregunta 4

Correcta

Puntúa 1,00 sobre 1,00

🚩 Marcar pregunta

- Una clase Hija para ejecutar los métodos de la clase Padre está obligada a anteponer la palabra "Super"

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✔️

La respuesta correcta es 'Falso'

Pregunta 5

Finalizado

Puntúa 3.00 sobre 3.00

 Marcar pregunta

Considere el siguiente fragmento de código:

```
/**
 * precondiciones:
 * sexo = 1, masculino
 * sexo = 2, femenino
 */
public int categoria(int edad, double estatura, int peso, int sexo) throws ...
{
    int rta = 1;
    int var1 = 0;
    double var2 = 0.0;
    double var3 = 0.0;
    //....
    if (edad >18 && edad < 60)
        var1 = 1;
    //....
    var2 = estatura * 1,25;
    //....
    var3 = sexo * peso * 0,25; //factor que influye a los hombres solamente
                               //y esta determinado por el peso.
    //TODO: código que resuelve
    return rta;
}
```

Determine qué excepciones debería lanzar que sean estrictamente necesarias para que este código funcione. Bosqueje el código del método categoría() con la lista de excepciones y el código interno de cada lanzamiento.

Rediseñe el método y el contrato, eligiendo qué precondiciones debería poner y qué excepciones lanzar. Bosqueje el código y el contrato.

```
/**
 * precondiciones:
 * sexo = 1, masculino
 * sexo = 2, femenino
 */
public int categoria(int edad, double estatura, int peso, int sexo) throws PesoInvalidoException, EstaturaInvalidaException, EdadInvalidaException
{
    int rta = 1;
    int var1 = 0;
    double var2 = 0.0;
    double var3 = 0.0;
    if (edad > 0) {
        if (edad > 18 && edad < 60)
            var1 = 1;
        if (estatura > 0) {
            var2 = estatura * 1,25;
            if (peso > 0)
                if (sexo == 1)
                    var3 = sexo * peso * 0,25; //factor que influye a los hombres solamente
                    //y esta determinado por el peso.
                else
                    throw new PesoInvalidoException("El peso debe ser positivo");
        }
        else
            throw new EstaturaInvalidaException("La estatura debe ser positiva");
    }
    //TODO: código que resuelve
    }
    else
        throw new EdadInvalidaException("La edad debe ser positiva");
    return rta;
}
```

```

/**
 * precondiciones:
 * sexo = 1, masculino
 * sexo = 2, femenino
 * estatura > 0
 * edad > 0
 * peso > 0
 */
public int categoria(int edad, double estatura, int peso, int sexo)
{
    int rta = 1;
    int var1 = 0;
    double var2 = 0.0;
    double var3 = 0.0;
    if (edad >18 && edad < 60)
        var1 = 1;
    var2 = estatura * 1.25;
    if (sexo==1)
        var3 = sexo * peso * 0.25; //factor que influye a los hombres solamente
                                   //y esta determinado por el peso.

    //TODO: código que resuelve
    return rta;
}

```

Comentario:

Buen diseño, dado por válido.

Como sugerencia, se podría haber verificado también el límite superior de los valores, o sea, edad<120, estatura >2,5 metros, y así.

Igual vale completo

Pregunta **6**

Incorrecta

Puntúa 0,00 sobre 1,00

🚩 Marcar pregunta

Una clase concreta puede tener una clase hija abstracta

Seleccione una:

☐ Verdadero

☒ Falso ❌

La respuesta correcta es 'Verdadero'

Pregunta 7

Correcta

Puntúa 1,00 sobre 1,00

🚩 Marcar pregunta

Mire el siguiente código

```
class Scoop
{
    static int thrower() throws Exception
    { return 42; }

    public static void main(String [] args)
    {
        try
        {
            int x = thrower();
        } catch (Exception e)
        {

            x++;
        }
        finally
        {
            System.out.println("x = " + ++x);
        }
    }
}
```

Responda si la salida esperada es "falla de compilación"

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

Pregunta 8

Correcta

Puntúa 1,00 sobre 1,00

🚩 Marcar pregunta

El patrón Singleton permite tener una única instancia de una clase y esta instancia no puede ser usada por otras clases

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es 'Falso'

Pregunta 9

Correcta

Puntúa 1,00 sobre 1,00

🚩 Marcar pregunta

Una clase derivada sólo puede acceder a un método privado de la superclase si y sólo si ambas pertenecen al mismo paquete.

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

Falso, una clase derivada no puede acceder a los métodos privados de la superclase, sólo la misma clase tiene acceso a sus métodos privados.

La respuesta correcta es 'Falso'

Pregunta 10
Finalizado
Puntuación 0,00 sobre 3,00
🚩 Marcar pregunta

Considere una clase abstracta (Ordenable) que realice la ordenación de un arreglo compuesto por instancias de una clase de extienda de Ordenable:

```
public abstract class Ordenable
{
    public void ordenar_selección(Ordenable[] b) { .....}
    public abstract int compare(Ordenable b);
    public int p_menor(Ordenable[] b) { .....}
    public void intercambio( Ordenable[] b, int i, int j) {.....}
}
```

Indique esquematizando de la misma forma, cómo resolvería esto mismo utilizando una interface Comparable, que contenga el método abstracto compareTo(Object o).
Escriba también el método main() que realice la invocación.

```
public abstract class Comparable
{
    public void ordenar_selección(Comparable[] b) { .....}
    public abstract int compareTo(Comparable b);
    public int p_menor(Comparable[] b) { .....}
    public void intercambio( Comparable[] b, int i, int j) {.....}
}

public class Numero implements Comparable{
    public Numero() { //Constructor
        ...
    }
    ... //metodos y sobreescritura de compareTo
}

public class Prueba {

    public static void main(String[] args) {
        Comparable num = new Numero();
        //Metodos
    }
}
```

Comentario:
Mal, confunde conceptos. No sabe que Comparable es parte de Java

Pregunta 11
Correcta
Puntuación 1,00 sobre 1,00
🚩 Marcar pregunta

El patrón Singleton permite tener una única instancia de una clase a la que sólo puede acceder el primer método que la llama.

Seleccione una:

- ☐ Verdadero
- ☒ Falso ✓

La respuesta correcta es 'Falso'

Pregunta 12
Correcta
Puntuación 1,00 sobre 1,00
🚩 Marcar pregunta

Un método que propaga excepciones si se sobrescribe puede eliminar esa posibilidad

Seleccione una:

- ☒ Verdadero ✓
- ☐ Falso

La respuesta correcta es 'Verdadero'

Pregunta 13

Finalizado

Puntúa 1,50 sobre 3,00

🚩 Marcar pregunta

Indique qué debería considerar para que la clase Acorazado se pueda clonar y desarrolle el método Clone() para hacer una clonación correcta. Considerar que la clase Armamento_cañon y Armamento_Misil solo se pueden clonar si su posición es babor o estribos, pero nunca si están en popa o proa.

```
Barco
+String Nombre;
+Float Velocidad;
+Armamentos Armamento;

Armamento <abstracta>
+String Descripción;
+Float Alcance;
+enum Posiciones { POPA, BABOR, ESTRIBOR, PROA };

Armamento_Misil
+double Potencia_explosivo;
+boolean ILS;
+posicion Posiciones;

Armamento_Cañon
+int Cargador;
+posicion Posiciones;
```

Acorazado debe ser hija de barco y barco debe ser clonable

@Override

```
public Object clone() throws CloneNotSupportedException {

    Acorazado acorazadoClonado= null;
    acorazadoClonado= (Acorazado) super.clone();
    if (this.armamento_Misil !=null && this.armamento_Misil!="proa" && this.armamento_Misil !=popa)
        acorazadoClonado.armamento_Misil = (Armamento_Misil) this.armamento_misil.clone();

    if (this.armamento_Cañon !=null && this.armamento_Cañon.posicion !="proa" && this.armamento_Cañon.posicion != "popa")
        acorazadoClonado.armamento_Misil = (Armamento_Misil) this.armamento_misil.clone();

    return acorazadoClonado;
}
```

Comentario:

El método clone() de Acorazado debería tener Try-Catch y delegar a Armamento la clonación,.

Además la clase Armamento debería implementar Cloneable

Pregunta 14

Correcta

Puntúa 1,00 sobre 1,00

🚩 Marcar pregunta

El mejor lugar para ubicar la zona de recuperación de errores (bloque catch()) siempre es en el mismo método en donde se creó y lanzó la excepción correspondiente.

Seleccione una:

☐ Verdadero

☒ Falso ✓

La respuesta correcta es 'Falso'